DHANASEKARAN D
www.linkedin.com/in/dhanasekarand63

**1. How can you read different types of files in Azure Databricks using Spark?**

You can read files in Databricks using spark.read. You specify the file format and additional options as needed. Common formats include **CSV, JSON, Parquet, and text**. For example:

```
# Read a text file
df_text = spark.read.text("/path/to/file.txt")

# Read a CSV file with header
df_csv = spark.read.option("header", "true").csv("/path/to/file.csv")

# Read a JSON file
df_json = spark.read.format("json").load("/path/to/file.json")

# Read a Parquet file
df_parquet = spark.read.parquet("/path/to/file.parquet")
```

**2. Can you explain the difference between a job cluster and an interactive cluster in Databricks?**

- **Job Cluster:**
  - Created **temporarily** to run a specific job.
  - Automatically **terminated** after the job completes.
  - Optimized for **automated or scheduled workflows**.
- **Interactive Cluster:**
  - **Always running** and available for use.
  - Used for **development, testing, and ad hoc analysis** via notebooks.
  - Supports **iterative work** where you need to interact with data frequently.

**3. What is Azure Databricks, and how does it work with other services in the Azure ecosystem?**

Azure Databricks is a **fast, easy, and collaborative Apache Spark-based analytics platform** fully optimized for Microsoft Azure. It provides a unified environment for **data engineering, data science, and analytics**.

Key integrations with Azure services include:

- **Azure Active Directory (AAD):** Secure authentication and role-based access control.
- **Azure Blob Storage & Azure Data Lake Storage (ADLS):** Scalable and secure storage for data.
- **Azure Synapse Analytics:** Advanced analytics and data warehousing.

- **Azure Machine Learning (AML):** Building, training, and deploying machine learning mode

**4. Can you explain what Change Data Capture (CDC) is and how it is used in data pipelines?**

**Change Data Capture (CDC)** is a technique that **detects and captures only changes**—such as **INSERT, UPDATE, and DELETE**—in source data instead of reloading the full dataset. It enables **incremental data loading**, which improves performance and reduces storage and processing costs.

**Common use cases:**

- Real-time ETL pipelines for up-to-date data processing.
- Replicating transactional databases to a data warehouse or data lake.
- Streaming ingestion using tools like **Debezium** or **Azure Data Factory**

**5. What are magic commands in Databricks, and when would you use them?**

**Magic commands** in Databricks allow you to execute **specific actions or languages** within a notebook cell, regardless of the notebook's default language.

**Common magic commands and their uses:**

- %python, %scala, %r, %sql: Run code in a specific language in that cell.
- %sh: Run shell commands from within a cell.
- %fs: Access and manage Databricks File System (DBFS) commands.
- %md: Render cell content as **Markdown** for documentation or notes.
- %run: Execute another notebook from the current notebook.

**6. Can you explain the Databricks Lakehouse architecture and the purpose of the Bronze, Silver, and Gold layers?**

The **Databricks Lakehouse architecture** organizes data into **three layers** to support scalable, reliable, and efficient analytics:

- **Bronze Layer:**
  - Raw data ingested from source systems with **minimal transformations**.
  - Stores **original, unprocessed data** for auditing and traceability.
- **Silver Layer:**
  - Data is **cleaned, validated, and enriched**.

  - Ensures **quality and consistency** for downstream analytics.

- **Gold Layer:**
  - **Aggregated and curated data** for reporting, BI dashboards, and analytics.
  - Optimized for **business-level insights** and decision-making

**7. Can you describe the key components of Azure Databricks architecture?**

The **Azure Databricks architecture** consists of several key components that enable collaborative, scalable, and automated data processing:

- **Workspace:** Collaborative environment to manage **notebooks, libraries, and dashboards**.
- **Clusters:** Managed **Apache Spark clusters** that can be dynamically scaled based on workload.
- **Jobs:** Automate workflows to run **notebooks, JARs, or Python scripts**.
- **Databricks File System (DBFS):** A **distributed file system abstraction** over Azure Blob Storage, making data accessible for processing.

**8. How does Azure Databricks ensure data security and maintain compliance?**

Azure Databricks provides **robust data security and compliance** through several mechanisms:

- **Secure Authentication:** Integration with **Azure Active Directory (AAD)** for user authentication and **role-based access control**.
- **Data Encryption:** Encrypts data **at rest and in transit** using industry-standard protocols.
- **Network Security:** Supports **Virtual Network (VNet) injection** to isolate clusters in a private network.
- **Compliance Certifications:** Meets standards like **GDPR, HIPAA, and SOC 2** for regulatory compliance.

**9. Why is it not recommended to use inferred schema for large datasets in Databricks?**

Using **inferred schema** means Spark/Databricks reads multiple rows to **automatically detect column data types**. While convenient, it has drawbacks for large datasets:

**Issues with inferred schema:**

- **Performance Overhead:** Reading large files to infer types increases processing time.

- **Inaccuracy:** Sampling may miss edge cases, resulting in **incorrect data types**.

- **Scalability:** Slows down pipeline initialization and affects large-scale data processing.

**10. How do you implement data partitioning in Databricks, and when is it necessary to repartition data?**

**Data partitioning** in Databricks is used to **organize large datasets into smaller, manageable chunks** to improve query performance and parallelism.

**How to implement partitioning:**

- While writing a DataFrame to storage (e.g., Parquet), specify **partition columns**:

df.write.partitionBy("column_name").parquet("/path/to/output")

- While reading, Spark can **prune partitions** for faster queries.

**When to repartition:**

- When the data is **skewed**, or the number of partitions is **too small or too large** for the cluster.
- Before **joins or aggregations** to optimize parallelism and reduce shuffle.
- Use df.repartition(n) to **increase partitions** or df.coalesce(n) to **decrease partitions** efficiently.

**11. How can you execute one Databricks notebook from another notebook?**

You can call or execute a notebook from another notebook using the **%run magic command**. This is useful for **sharing helper functions, reusable code, or setup scripts** across multiple notebooks.

**Usage example:**

%run "/path/to/other_notebook"

- %run must be placed in a **cell by itself**.

- The notebook runs **inline**, making its functions and variables available in the calling notebook.

**12. How can you copy files from Azure Blob Storage to Databricks?**

You can copy files from **Azure Blob Storage** to Databricks using **dbutils.fs.cp** or by first **mounting the Blob Storage**.

**Option 1: Direct copy using dbutils.fs.cp**

```
dbutils.fs.cp(
  "wasbs://<container>@<account>.blob.core.windows.net/<path>",
  "dbfs:/mnt/<mount-name>/<path>",
  recurse=True
)
```

**Option 2: Mount Blob Storage first and access like DBFS**

```
dbutils.fs.mount(
  source = "wasbs://<container>@<account>.blob.core.windows.net",
  mount_point = "/mnt/<mount-name>",
  extra_configs = {"<conf-key>": "<conf-value>"}
)
```

- Once mounted, you can **read/write files directly** using DBFS paths

**13. What cluster modes are available in Azure Databricks, and when should you use each mode?**

- **Standard Mode:**
  - Suitable for **most workloads**, balancing **performance and cost**.
  - Commonly used for **data engineering, batch processing, and analytics**.

- **High Concurrency Mode:**
  - Optimized for **multiple users accessing the same cluster concurrently**.
  - Ideal for **BI dashboards, shared notebooks, and interactive workloads**.

- **Single Node Mode:**
  - Runs all Spark components on a **single node**.
  - Best for **lightweight development, testing, or debugging** small datasets.

**14. What techniques can you use to optimize the performance of a Spark job in Azure Databricks?**

- **Data Partitioning:** Partition data appropriately to enable **parallel processing** and reduce **shuffle operations**.

- **Caching:** Cache frequently accessed data using df.cache() to **avoid recomputation**.

- **Cluster Configuration:** Choose the right **cluster size and type** based on the workload and data volume.

- **Broadcast Variables:** Use **broadcast variables** to efficiently distribute small datasets to all nodes and reduce **network overhead**.

**15. What is a prolonged secrets scope in Databricks, and how is it used?**

DHANASEKARAN D
www.linkedin.com/in/dhanasekarand63

A **prolonged secrets scope** in Databricks refers to **secrets (credentials, tokens, or keys) that are stored for long-term or repeated access**. It allows secure, reusable access to sensitive information without hardcoding it in notebooks or pipelines.

**Types of secret scopes supported:**

- **Databricks-backed scopes:** Managed directly by Databricks.
- **Azure Key Vault-backed scopes:** External integration for enhanced security.

**Use cases:**

- Securely mounting **Azure Data Lake Storage (ADLS)**.
- Accessing **APIs or databases** without exposing credentials.
- Reusable secrets across **pipelines and notebooks**.

**16. How would you debug complex Spark applications in Azure Databricks?**

- **Use Spark UI:** Inspect **jobs, stages, and tasks** to identify slow stages or failed tasks.
- **Check Driver and Executor Logs:** Review **stdout, stderr, and event logs** for errors or warnings.
- **Unit Testing & Small Data Samples:** Test transformations on **small datasets** before running full-scale jobs.
- **Structured Logging:** Implement **logging within notebooks or scripts** to track data flow and variable values.
- **Break Down Pipelines:** Divide large jobs into **smaller, manageable steps** to isolate issues.
- **Cluster Configuration Checks:** Ensure clusters have **enough resources** and appropriate Spark configurations.

**17. How do you create a cluster in Azure Databricks?**

To create a cluster in Databricks:

1. Navigate to the **Compute** tab in your Databricks workspace.
2. Click **Create Cluster**.
3. Select the **cluster type** based on your workload (e.g., Standard, High Concurrency, Single Node).
4. Configure cluster options such as **Spark version, worker nodes, and autoscaling**.

5. Click **Create** to start the cluster.

**18. How do you create and deploy notebooks in Azure Databricks?**

**Creating a notebook:**

1. Navigate to the **Databricks workspace**.
2. Click **New → Notebook**.
3. Choose the **language** (Python, Scala, SQL, or R) and attach it to a cluster.

**Deploying a notebook:**

- Use **Jobs or Workflows** to **schedule and trigger execution**.
- Integrate with **CI/CD pipelines** for automated deployment and version control.
- Notebooks can be executed manually or as part of **ETL/ML pipelines.**

**19. What is Delta Lake, and how does it improve data reliability in Azure Databricks?**

**Delta Lake** is an open-source storage layer that brings **ACID (Atomicity, Consistency, Isolation, Durability) transactions** to Apache Spark and big data workloads. In Azure Databricks, it enhances **data reliability** through:

- **ACID Transactions:** Ensures **data consistency** during concurrent reads and writes.
- **Schema Enforcement:** Prevents ingestion of data that **doesn't match the defined schema**, reducing errors.
- **Time Travel:** Allows access to **previous versions of data** for auditing, debugging, or rollback purposes.

**19. How do you manage version control and collaboration in Azure Databricks?**

Azure Databricks provides several features to support **version control and collaboration**:

- **Databricks Repos:** Integrates with **Git repositories** (GitHub, Azure DevOps) to version notebooks and code.
- **Collaborative Notebooks:** Allows **real-time co-authoring**, commenting, and access to **revision history**.
- **Permissions Management:** Provides **granular access control** to manage who can view or edit notebooks and workspace assets.

**20. What parameters are required when mounting storage in Azure Databricks, and how should you handle credentials?**

To mount **Azure Data Lake Storage (ADLS) or Blob Storage** in Databricks, you use dbutils.fs.mount() with the following parameters:

```
dbutils.fs.mount(
  source = "wasbs://<container>@<storage-account>.blob.core.windows.net/",
  mount_point = "/mnt/<mount-name>",
  extra_configs = {
    "fs.azure.account.key.<storage-account>.blob.core.windows.net": "<access-key>"
  }
)
```

**Parameters explained:**

- **source:** URL of the storage container.
- **mount_point:** Path in Databricks File System (DBFS) where the storage will be accessible.
- **extra_configs:** Authentication information such as **account key, SAS token, or OAuth** credentials.
-

**Best practice:**

- Store secrets in **Databricks secret scopes** instead of hardcoding them in notebooks.

**21. How can you implement a CI/CD pipeline for Azure Databricks notebooks?**

To implement a **CI/CD pipeline** for Databricks notebooks:

1. **Version Control:** Store notebooks and related code in a **Git repository** (GitHub, Azure DevOps, etc.).

2. **Branching Strategy:** Use branches for **development, testing, and production** workflows.

3. **Automated Testing:** Implement **unit tests or data quality checks** on notebooks or scripts.

4. **CI/CD Integration:**
   - Use **Azure DevOps, GitHub Actions, or Jenkins** to automate deployment.
   - Deploy notebooks using **Databricks CLI**, **REST API**, or **Databricks Repos**.

5. **Workflows/Jobs:** Configure **Databricks Jobs or Workflows** to schedule and execute notebooks in production.

**22. How can you view the list of files in Databricks, both via commands and the UI?**

You can view files in Databricks using multiple methods:

**1. Using dbutils.fs command:**

```
dbutils.fs.ls("/path/to/directory")
```

**2. Using %fs magic command:**
%fs ls /path/to/directory

**3. Using the Databricks UI:**

- Navigate to the **Data tab** in your workspace.
- Browse **DBFS locations** to view files and directories interactively.

**23. What is Unity Catalog in Databricks, and what are its key features?**

**Unity Catalog** is a **centralized governance layer** in Databricks for managing data access, lineage, auditing, and discovery across all workspaces.

**Key features:**

- **Centralized Data Governance:** Manage data access consistently across all Databricks workspaces.
- **Fine-Grained Access Control:** Supports **table-, column-, and row-level permissions**.
- **Audit & Lineage:** Track **who accessed what data** and maintain **data lineage** for compliance.
- **Data Discovery:** Simplifies finding and understanding datasets within the organization

**24. What are Databricks Jobs, and how are they used for workflow automation?**

**Databricks Jobs** are used to **schedule and automate the execution of code-based workflows**. They help streamline repetitive tasks and orchestrate data pipelines.

**Key uses:**

- **Notebook Execution:** Run notebooks automatically on a **scheduled basis**.
- **JAR and Python Scripts:** Execute **compiled JAR files or Python scripts** as part of a workflow.
- **Parameterized Runs:** Pass parameters to jobs for **dynamic execution** and flexible workflows.

**25. How does Azure Databricks work with Azure Machine Learning for model development and deployment?**

Azure Databricks integrates with **Azure Machine Learning (AML)** to support the full **machine learning lifecycle**:

- **Model Development:** Databricks provides **scalable Spark clusters** for **data preparation, feature engineering, and model training**.

- **Model Registration:** Trained models can be **registered directly in the AML Model Registry** from Databricks.

- **Model Deployment:** Models can be deployed to **AML endpoints** for **real-time scoring** or **batch inference**.

**26. Does Databricks support stored procedures, and how can you implement procedural logic?**

Databricks does **not natively support traditional stored procedures** like SQL Server. However, you can implement procedural or parameterized logic using other methods:

- **Notebooks or Delta Live Tables:** Write procedural logic in **Python, Scala, or SQL**.

- **User-Defined Functions (UDFs) & SQL Widgets:** Build **parameterized operations** for reusability.

- **SQL Dashboards with Filters:** Apply logic through **interactive dashboards**.

**27. How can you connect Azure Data Lake Storage (ADLS) to Databricks?**

You can connect **ADLS** to Databricks using **two main approaches**:

**1. Direct Access:**

- Read data directly using Spark without mounting:

```
df = spark.read.format("parquet").load(
  "abfss://<container>@<storage-account>.dfs.core.windows.net/<path>"
)
```

**2. Mount ADLS to DBFS:**

- Mount ADLS using **service principal** or **managed identity** for seamless access:

```
dbutils.fs.mount(
  source = "abfss://<container>@<storage-account>.dfs.core.windows.net/",
  mount_point = "/mnt/<mount-name>",
  extra_configs = {"<conf-key>": "<conf-value>"}
)
```

- After mounting, you can access files via **/mnt/<mount-name>** paths.

**28. How can you implement data versioning in Azure Databricks to track historical changes?**

In **Azure Databricks**, data versioning is implemented using **Delta Lake's Time Travel** feature. Delta Lake maintains a **transaction log** that records all changes made to a table, allowing you to query **previous versions** of data.

**Usage examples:**

- Query by version number:

```
SELECT * FROM my_table VERSION AS OF 3;
```

- Query by timestamp:

```
SELECT * FROM my_table TIMESTAMP AS OF '2023-10-01';
```

**Benefits:**

- **Auditing:** Track who changed data and when.
- **Reproducibility:** Reproduce experiments using historical data.
- **Rollback:** Restore data to a previous state if needed.

**29. What is the role of Azure Databricks in a modern data architecture?**

**Azure Databricks** acts as a **unified analytics platform** in modern data architectures, bridging the gap between **data engineering, analytics, and data science**.

**Key roles:**

- **Data Ingestion:** Efficiently ingest **batch and streaming data** from multiple sources.
- **Data Processing:** Perform **large-scale transformations and aggregations** using **Apache Spark**.
- **Machine Learning:** Develop, train, and deploy **ML models** within the same environment.
- **Collaboration:** Enable **data engineers, data scientists, and analysts** to collaborate using **shared notebooks and integrated workflows**.

**30. How can you configure and manage job retries in Azure Databricks?**

In **Azure Databricks**, you can manage job retries to ensure **reliable execution of workflows** in case of transient failures:

- **Retry Settings:** When creating or editing a job, you can configure the **number of retry attempts** and **retry interval** in the job configuration.
- **Failure Handling:** If a task fails, Databricks will **automatically retry** it based on the configured settings.
- **Dependent Tasks:** Jobs with **task dependencies** can be configured to retry failed tasks without affecting successful tasks in the workflow.

- **Monitoring:** Use **job run history and alerts** to monitor retries and failures for troubleshooting.

**31. What is the difference between a Data Lake and Delta Lake in Azure Databricks?**

| Feature | Data Lake | Delta Lake |
|---|---|---|
| **Storage Type** | Stores raw data in **append-only mode**. | Stores data with **ACID transactional support**, allowing updates and deletes. |
| **Data Reliability** | No built-in transactional guarantees; prone to data inconsistencies. | **ACID transactions** ensure **data consistency and reliability**. |
| **Time Travel / Versioning** | Not supported. | Supports **Time Travel** to query previous versions of data. |
| **Use Case** | Good for **storing large amounts of raw data**. | Suitable for **reliable ETL pipelines, auditing, and incremental update** |

**32. What are Delta logs in Delta Lake, and how can you track data versioning in Delta tables?**

**Delta logs** are transaction logs that Delta Lake maintains in the _delta_log/ directory of a table. They store **JSON files** that record all changes (inserts, updates, deletes) to the table.

**Tracking data versioning:**

- Use the **DESCRIBE HISTORY** command to view the table's version history:

DESCRIBE HISTORY delta.`/path/to/table`

- This shows:
  - **Version numbers**
  - **Timestamps of operations**
  - **Operation types** (e.g., MERGE, UPDATE, DELETE)
  - **User who performed the operation**

**33. How can you monitor and troubleshoot performance issues in Azure Databricks?**

- **Spark UI:** Provides detailed insights into **job execution**, including **stages, tasks, and resource usage**.

- **Ganglia Metrics:** Monitors **cluster-level metrics** such as CPU, memory, and disk utilization.

- **Databricks Notebooks:** Enable **custom logging and visualization** of metrics during job execution.

- **Azure Monitor Integration:** Allows **centralized logging, alerts, and dashboards** across Azure services, including Databricks.

**34. What factors should you consider when scaling Azure Databricks clusters for large datasets?**

When scaling Databricks clusters to handle large datasets, consider the following:

- **Cluster Size:** Increase the number of **worker nodes** based on data volume and processing needs.
- **Auto-Scaling:** Enable **auto-scaling** to dynamically add or remove nodes based on workload demands.
- **Instance Types:** Choose VM sizes that provide the **best balance between performance and cost**.
- **Data Partitioning:** Partition data effectively to **maximize parallel processing** and minimize shuffles.

**35. How can you securely manage secrets in Azure Databricks when accessing external data sources**

In Azure Databricks, secrets can be managed securely using **Databricks Secret Scopes**:

- **Secret Scopes:** Create scopes to store **credentials, API keys, tokens, or passwords**.
- **Access Secrets Programmatically:** Use dbutils.secrets.get("scope-name", "secret-key") to retrieve secrets without hardcoding them in notebooks.
- **Integration with Key Vault:** You can back secret scopes with **Azure Key Vault** for **centralized enterprise-grade secret management**.
- **Fine-Grained Access Control:** Grant **read access to specific users or groups**, ensuring only authorized users can access secrets.

**36. What is a Hive Metastore, and what role does it play in Azure Databricks?**

The **Hive Metastore** is a **relational database** that stores metadata about tables and schemas in Spark. It enables easy querying of large datasets by keeping track of **table structure and schema information**.

**Key points:**

- When a Spark table is created, its **schema details** (column names, data types) are stored in the Hive Metastore.

- Every **Databricks deployment** has a **central Hive Metastore** accessible by all clusters to persist table metadata.

- You can also choose to use an **external Hive Metastore** instead of the default Databricks one.

**37. How can you implement SCD Type 1 and Type 2 using PySpark in Azure Databricks**?

**Slowly Changing Dimensions (SCD)** can be implemented in Databricks using **PySpark and Delta Lake**:

**1. SCD Type 1 – Overwrite old records**

- Updates existing records with **new data**, without keeping history.
- Example using Delta Lake:

```
MERGE INTO target t
USING source s
ON t.id = s.id
WHEN MATCHED THEN UPDATE SET t.col1 = s.col1
WHEN NOT MATCHED THEN INSERT (id, col1) VALUES (s.id, s.col1)
```

**2. SCD Type 2 – Maintain history**

- Keeps historical versions using columns like **is_current, start_date, end_date**.
- Example using Delta Lake:

```
MERGE INTO target t
USING source s
ON t.id = s.id
WHEN MATCHED AND (t.col1 != s.col1) THEN
  UPDATE SET t.is_current = false, t.end_date = current_date()
WHEN NOT MATCHED THEN
  INSERT (id, col1, is_current, start_date, end_date)
  VALUES (s.id, s.col1, true, current_date(), null)
```

**38. How can you manage dependencies and libraries in Azure Databricks notebooks?**

Managing dependencies in Databricks notebooks can be done through several approaches:

1. **Cluster-Level Libraries:**

- Install libraries via the **Databricks UI** at the cluster level.
- Ensures **all notebooks** running on the cluster have access to the libraries.

2. **Notebook-Scoped Libraries:**
- Install libraries **only for a specific notebook** using magic commands:

```
%pip install <library-name>
%conda install <library-name>
```

- Isolates dependencies to that notebook without affecting other notebooks.

3. **Version Control:**
- Specify **library versions** to maintain consistency across clusters and environments.

**39. How can Azure Databricks be integrated with Azure DevOps for CI/CD pipelines?**

1. **Version Control:**
- Store **Databricks notebooks and code** in **Azure Repos** to track changes and collaborate effectively.

2. **Build Pipelines:**
- Use **Azure Pipelines** to **automate testing and validation** of Databricks code before deployment.

3. **Release Pipelines:**
- Automate deployment of notebooks, libraries, and configurations across **development, staging, and production environments**.

**40. How can you mask or protect sensitive data (e.g., PII) before sharing it across teams in Azure Databricks?**

1. **Delta Lake Features:**
- Use **column-level security** and **masking policies** in Delta tables to restrict access to sensitive columns.

2. **Data Masking / Obfuscation:**
- Apply **masking functions** or transformations in notebooks, e.g., replacing PII with hashed or anonymized values:

```
from pyspark.sql.functions import sha2, concat_ws

df_masked = df.withColumn("email_masked", sha2("email", 256))
```

3. **Unity Catalog:**
- Apply **fine-grained access control** to enforce who can view or query sensitive data.

4. **Best Practices:**
- Avoid sharing raw PII.
- Use **sandboxed environments** for development.
- Combine with **role-based access control (RBAC)** for secure access.

DHANASEKARAN D
www.linkedin.com/in/dhanasekarand63

**41. If you have developed a Databricks notebook that needs to run every hour, how would you automate its execution?**

In Azure Databricks, you can automate notebook execution using **Jobs/Workflows**:

1. **Create a Job:**
- Navigate to **Workflows → Jobs → Create Job** in the Databricks UI.

2. **Configure the Notebook Task:**
- Select the notebook you want to execute.
- Specify the **cluster** to run the notebook.

3. **Schedule the Job:**
- Set the **recurrence** to run **every hour** (or any desired interval).
- Optionally configure **retry policies** and **notifications** on failure.

**42. How does auto-scaling work in Azure Databricks clusters, and why is it useful?**

**Auto-scaling clusters** in Azure Databricks automatically **adjust the number of worker nodes** based on workload demand.

**Key points:**

- **Dynamic Scaling:** Clusters automatically **increase workers** when workload is high and **decrease workers** when demand drops.

- **Configuration:** You define **minimum and maximum number of workers** in the cluster settings.

- **Benefits:**
  - Optimizes **performance** by providing sufficient resources for heavy jobs.
  - Reduces **costs** by scaling down when resources are idle.

**43. How can you handle schema evolution in Delta Lake when working with Azure Databricks?**

Delta Lake in Azure Databricks supports **schema evolution**, which allows the data schema to change over time without breaking pipelines:

1. **Automatic Schema Evolution:**
- Enable the option mergeSchema = true when writing data.
- This allows **new columns to be added automatically** without manual intervention:

```
df.write.format("delta").mode("append").option("mergeSchema",
"true").save("/path/to/delta_table")
```

2. **Schema Enforcement:**

- Delta Lake **enforces the existing schema** to prevent accidental writes of incompatible data.
- Ensures **data integrity and consistency** across versions.

**44. What are the best practices for securing data and resources in Azure Databricks?**

To secure data and resources in Azure Databricks, follow these best practices:

1. **Data Encryption:**
- Encrypt data **at rest** using Azure Storage encryption.
- Encrypt data **in transit** using TLS/SSL protocols.

2. **Access Controls:**
- Implement **Role-Based Access Control (RBAC)** to restrict access based on user roles.
- Use **Unity Catalog** for fine-grained access to tables, columns, and rows.

3. **Network Security:**
- Use **Virtual Network (VNet) injection** to isolate Databricks clusters within a private network.
- Restrict **public access** and enforce **network security groups**.

4. **Audit Logging:**
- Enable **audit logs** to track access, configuration changes, and data modifications.
- Helps meet **compliance requirements** like GDPR, HIPAA, and SOC 2.

**45. As a Data Engineer at a company, if you need to reuse code developed by another team in an Azure Databricks notebook, how would you do it following best practices?**

1. **Import from the same workspace:**
- If the reusable code is available **within the same workspace**, you can directly **import notebooks or Python/Scala modules** using:

```
%run "/Workspace/Path/To/Notebook"
```

2. **Import external code (outside workspace):**
- Package the code as a **JAR (for Scala/Java)** or **Python module**.
- Attach the **JAR/module** to the cluster via **Libraries → Install New → Library from DBFS / Maven / PyPI**.
- Import it in your notebook using standard Python or Scala import statements.

46. What are some key optimization techniques you can apply in Azure Databricks to improve performance

1. **Use Delta Lake:**

- Provides **ACID transactions**, schema enforcement, and **faster reads/writes** compared to standard data lakes.

2. **Partition Large Tables:**
- Partitioning allows Spark to **read only relevant data**, reducing scan time and improving query performance.

3. **Z-Ordering:**
- Reorganize data within partitions using **Z-ordering** to improve **filtering performance** on multiple columns.

4. **Caching Frequently Accessed Data:**
- Use **df.cache()** or **spark.catalog.cacheTable()** to store hot data in memory, minimizing recomputation.

**47. How can you implement real-time or streaming data processing in Azure Databricks?**

1. **Structured Streaming:**
- Spark API that treats **streaming data as unbounded tables**.
- Enables **real-time transformations, aggregations, and analytics** on incoming data.

2. **Integration with Streaming Sources:**
- Connect to **Azure Event Hubs**, **Apache Kafka**, or other streaming platforms.
- Ingest streaming data directly into Databricks for **processing and analytics**.

**48. What are Databricks Delta Live Tables (DLT), and how do they simplify ETL processes in Azure Databricks?**

**Delta Live Tables (DLT)** is a framework in Azure Databricks designed to **simplify the creation, management, and monitoring of ETL pipelines**.

**Key Features and Benefits:**

1. **Simplified Pipeline Development:**
- Define ETL transformations **declaratively** using **SQL or Python**.
- Reduces the complexity of writing manual ETL code.

2. **Automated Pipeline Management:**
- DLT automatically handles **infrastructure provisioning**, **job scheduling**, and **error handling**.
- Ensures pipelines run **reliably at scale**.

3. **Data Quality Enforcement:**
- Supports **data quality rules** (expectations) to validate incoming data.
- Ensures only **clean and reliable data** is propagated downstream.

**49. How do you decide which cluster type and configuration to use in Azure Databricks for different workloads?**

1. **Cluster Type:**
- **Interactive Cluster:**
    - Best for **development, debugging, and ad hoc analysis** using notebooks.
    - Provides flexibility to **experiment and test**.
- **Job Cluster:**
    - Ideal for **production workloads and automated jobs**.
    - Created **temporarily** for a job and terminated after execution, optimizing cost.

2. **Cluster Configuration:**
- Decide the **number and type of nodes** based on workload:
    - **Memory-intensive workloads:** For jobs with heavy **joins, aggregations, or shuffles**, choose nodes with **more memory** to reduce shuffle operations.
    - **CPU-intensive workloads:** For computation-heavy tasks, scale **cores and workers** appropriately.
- Consider **auto-scaling** to dynamically adjust resources based on workload.

**50. How can you monitor and track the performance of jobs in Azure Databricks?**

1. **Jobs UI:**
- Provides **job status, run history, and execution logs**.
- Helps track **success/failure and execution times**.

2. **Spark UI and Logs:**
- Access **driver and worker logs** to debug errors or performance issues.
- View detailed **job stages and tasks** for troubleshooting.

3. **Cluster Metrics:**
- Monitor **CPU, memory, and disk usage** via the **Cluster Metrics tab**.
- Helps identify **resource bottlenecks** and optimize cluster configuration.

**51. How do you handle data skew in Azure Databricks to improve Spark job performance?**

**Data skew** occurs when certain partitions of data are much larger than others, causing **performance bottlenecks** in Spark jobs.

**Techniques to mitigate data skew in Azure Databricks:**

1. **Salting:**
- Add a **random suffix or prefix** to skewed keys to distribute data **evenly across partitions**.

2. **Repartitioning:**
- Repartition the dataset using a **more uniform key** to balance the load across workers:

```
df.repartition("new_partition_key")
```

3. **Broadcast Joins:**
- For **small tables**, use **broadcast joins** to send the smaller dataset to all nodes, reducing **shuffle operations**:

```
from pyspark.sql.functions import broadcast
df_large.join(broadcast(df_small), "key")
```

**52. What are the key considerations for implementing role-based access control (RBAC) in Azure Databricks?**

**Key considerations:**

1. **Workspace Permissions:**
- Assign roles such as **Admin, User, or Viewer** to control access to **notebooks, dashboards, and jobs**.

2. **Cluster Policies:**
- Define **cluster configuration restrictions** based on roles.
- Ensures **compliance, security, and cost management**.

3. **Data Access Controls:**
- Leverage **Azure Active Directory (AAD)** to manage access to **external data sources**.
- Ensure users have only the **permissions required** to perform their tasks.

**53. If multiple team members are working on the same Databricks notebook, how can you ensure that no one overwrites each other's work?**

1. **Git Integration:**
- Integrate Databricks notebooks with **Git repositories** (e.g., GitHub, Azure DevOps).
- Every change is **committed and pushed** to the repository, enabling **version control**.

2. **Branching and Merging:**
- Use **feature branches** for individual work.
- Merge changes through Git to **avoid conflicts** and maintain a clean main branch.

3. **Collaborative Features:**
- Databricks supports **real-time co-authoring** with **revision history and commenting**, which also helps track changes.

**54. How can you grant permissions to specific notebooks or clusters for different users in Azure Databricks?**

1. **Steps to Assign Permissions:**

- Navigate to the **notebook or cluster** in the workspace.
- Open the **Permissions tab**.
- Click **Add Users/Groups** and select the users or groups.
- Assign appropriate roles:
  - **Can View** – User can only view the notebook/cluster.
  - **Can Run** – User can execute the notebook or use the cluster.
  - **Can Edit** – User can modify the notebook.

**55. How do you implement incremental data processing in Azure Databricks for efficient ETL pipelines?**

Incremental data processing in Azure Databricks ensures only **new or changed data** is processed, improving performance and reducing costs.

**Techniques to handle incremental data processing:**

1. **Change Data Capture (CDC):**

- Capture only **INSERT, UPDATE, or DELETE** changes from the source since the last run.
- Minimizes the need to process full datasets.

2. **Delta Lake MERGE (Upsert):**

- Use **MERGE INTO** to efficiently **insert, update, or delete** records in Delta tables:

```
MERGE INTO target t
USING source s
ON t.id = s.id
WHEN MATCHED THEN UPDATE SET ...
WHEN NOT MATCHED THEN INSERT ...
```

3. **Watermarking (for Streaming):**

- Handle **late-arriving data** in streaming scenarios.
- Ensures **exactly-once processing** in Structured Streaming pipelines.

**56. How would you implement a Slowly Changing Dimension (SCD) Type 2 in Azure Databricks using Delta Lake?**

1. **Identify Changes:**

- Compare incoming records with the existing Delta table to detect **new and updated records**.

2. **Insert New Records:**

- Insert records that **don't exist** in the target table with a **current flag = true** and proper **start/end dates**.

3. **Expire Old Records:**

- For records that have changed, update the **current flag to false** and set the **end date** to indicate expiry.

4. **Insert Updated Records:**
- Insert the updated record as a **new entry** with **current flag = true**, **start date = today**, and **end date = null**.

**Example using Delta Lake MERGE:**

```
MERGE INTO target_dim t
USING source_data s
ON t.id = s.id
WHEN MATCHED AND t.col1 != s.col1 THEN
  UPDATE SET t.current_flag = false, t.end_date = current_date()
WHEN NOT MATCHED THEN
  INSERT (id, col1, start_date, end_date, current_flag)
  VALUES (s.id, s.col1, current_date(), null, true)
```

**57. Can you call a Databricks notebook from Azure Data Factory (ADF)? If yes, how would you do it?**

Yes, you can call a **Databricks notebook from Azure Data Factory** to orchestrate ETL or data processing workflows.

**Steps to achieve this:**

1. **Create a Databricks Linked Service in ADF:**
- Connect ADF to your Azure Databricks workspace using a **Databricks token or Azure authentication**.

2. **Create a Pipeline in ADF:**
- Design a pipeline where the Databricks notebook will be executed.

3. **Add a Databricks Notebook Activity:**
- Drag the **Databricks Notebook activity** into the pipeline.
- Configure the **notebook path, parameters (if any), and cluster settings**.

4. **Trigger the Pipeline:**
- Schedule or trigger the pipeline to run the notebook as part of your ETL workflow.

**58. How do you implement data validation and quality checks in Azure Databricks?**

Data validation and quality checks in Azure Databricks ensure that **data is accurate, consistent, and reliable** before further processing.

**Approaches to implement data quality checks:**

1. **Using Assertions:**
- Apply **assert statements** on DataFrames to validate conditions:

```
assert df.filter("salary < 0").count() == 0, "Negative salary found!"
```

- Useful for **quick validations** during development or ETL runs.

2. **Using Data Quality Libraries:**
- **Deequ (by AWS)**: Allows **automated data profiling and validation rules** on large datasets.
- **Great Expectations**: Provides a framework to define **expectations** (rules) for data quality and validate datasets against them.

3. **Integrating into Pipelines:**
- Embed these checks in **Databricks notebooks or workflows** to **fail jobs on invalid data**.
- Ensures **consistent enforcement of business rules** and **prevents bad data from propagating**.

**59. If you have a Databricks notebook and are required to write the resultant DataFrame to Snowflake, how would you handle this?**

1. **Indirect Approach via ADLS and Snowpipe/ADF:**
- Write the DataFrame to a **common storage location** like **Azure Data Lake Storage (ADLS)** in a format such as **Parquet or CSV**.
- Use **Snowpipe** or an **Azure Data Factory (ADF) pipeline** to **ingest the data into Snowflake**.
- Useful when **decoupling Databricks processing and Snowflake loading** or handling large batch jobs.

2. **Direct Approach using Snowflake Connector:**
- Use the **Databricks Snowflake connector** to write data directly from the notebook:

```
df.write \
 .format("snowflake") \
 .options(**snowflake_options) \
 .mode("overwrite") \
 .save()
```

- Snowflake options include **account, user, password, database, schema, and warehouse**.
- Efficient for **real-time or near-real-time data loading**.

**60. How can you manage and automate ETL workflows using Azure Databricks Workflows?**

**Steps to manage and automate ETL workflows:**

1. **Create a Workflow (Job):**
- Use the **Workflows/Jobs UI** in Databricks to create a new job.

2. **Define Tasks:**
- Add **tasks**, which can be **notebooks, JARs, Python scripts, or SQL scripts**.
- Tasks can be **chained or run in parallel** depending on dependencies.

3. **Configure Schedule & Retries:**
- Define **schedules** to run jobs automatically (e.g., hourly, daily).
- Set **retry policies** in case of task failures.

4. **Parameterize Workflows:**
- Pass **parameters** to notebooks or scripts for dynamic execution.
- Supports **environment-specific configurations** for dev/staging/prod.

**61. What are the best practices for securing data in Azure Databricks?**

**Best practices for securing data:**

1. **Access Control & Governance:**
- Use **Unity Catalog** and **Access Control Lists (ACLs)** for **fine-grained permissions** on tables, views, and notebooks.
- Manage **workspace and cluster access** through **role-based access control (RBAC)**.

2. **Authentication & Identity Management:**
- Enable **Credential Passthrough** or use **Managed Identity** to ensure **secure access to storage and data sources**.

3. **Network Security:**
- Enable **IP access control** to restrict workspace access.
- Use **secure cluster connectivity** (VNet injection) to isolate clusters within a private network.

4. **Data Encryption:**
- Encrypt data **at rest** using **Azure Storage encryption**.
- Encrypt data **in transit** using **TLS/SSL** protocols.

**62. If you create a Spark SQL table in Databricks and load data into it, but the cluster is terminated by mistake, will the table still be accessible? Explain why.**

Yes, the table will still be accessible even if the cluster is terminated.

**Reasoning:**

1. **Data Storage:**
- The actual data is stored in **external storage** (e.g., **Azure Data Lake Storage, Blob Storage, or DBFS**) and **not inside the cluster**.
- Terminating the cluster does **not delete the underlying data**.

2. **Schema Storage:**
- Table schema and metadata are stored in the **Hive Metastore** (Databricks-managed or external).
- Hive Metastore maintains information like **table name, columns, and data types**, independent of cluster runtime.

**63. How can you integrate Azure Databricks with Azure DevOps for implementing CI/CD pipelines?**

**Steps and best practices:**

1. **Version Control:**

- Store Databricks notebooks and code in a **Git repository** (e.g., **Azure Repos**).
- Enables collaboration, change tracking, and rollback of notebook versions.

2. **Automated Deployment:**

- Use **Databricks CLI or REST API** to deploy notebooks, libraries, and configurations from DevOps pipelines.
- Ensures consistent deployment across **development, staging, and production environments**.

3. **CI/CD Tools:**

- Example workflow: **Azure Pipelines + Databricks CLI + Terraform**
  - **Build pipeline:** Test notebooks, validate syntax, run unit tests.
  - **Release pipeline:** Deploy notebooks, configure clusters, and schedule jobs automatically.

**64. How can you use Databricks Delta Live Tables (DLT) for continuous data processing?**

Delta Live Tables (DLT) in Databricks provides a **declarative ETL framework** for building **reliable, continuous data pipelines**.

**Key features and usage:**

1. **Declarative Pipeline Definition:**

- Define **LIVE TABLES** using **SQL or Python**, specifying transformations and sources.
- Databricks automatically manages **execution, scaling, and monitoring**.

2. **Continuous Processing:**

- Supports **incremental data loads**, processing only new or updated data.
- Maintains **data quality** with built-in expectations and validation rules.
- Tracks **data lineage** for auditing and traceability.

3. **Example:**

```
CREATE LIVE TABLE bronze AS
SELECT * FROM cloud_files("/mnt/data/bronze", "json")
```

- This reads new JSON files from the cloud location and continuously updates the **bronze table**.

**65. How can you stop or halt a Databricks process that is already running?**

In Azure Databricks, there are multiple ways to stop a running process depending on the scope:

DHANASEKARAN D
www.linkedin.com/in/dhanasekarand63

1. **Stop a Notebook Cell:**
- If a specific cell is executing, click the **“Cancel” button** next to the running cell to halt execution.

2. **Terminate a Job:**
- For a scheduled or manually triggered job, navigate to the **Jobs UI**, locate the **active run**, and click **“Cancel Run”**.

3. **Kill a Spark Job/Stage:**
- Use the **Spark UI** to inspect running jobs or stages and cancel specific tasks if needed.

4. **Terminate the Cluster:**
- As a last resort, shutting down the cluster will stop **all active processes** on that cluster.
- Useful if a job is unresponsive or stuck in an infinite loop.

DHANASEKARAN D
www.linkedin.com/in/dhanasekarand63